



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingenieria de Sistemas

Proyecto Simulador de Bases de Datos

Curso: Calidad y Pruebas de Software

Docente: MAG. Patrick Cuadros Quiroga

Integrantes:

- **Jhony Vargas Luque (2022075754)**
- **Abel Fernando Pacompia Ortiz (2023076797)**

Tacna - Peru

2026

Documento de Vision

Version: 2.1

Version	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
--:	--:	--:	--:	--:	--
1.0	APO, JVL	APO, JVL	P. Cuadros Q.	2026-04-14	Version inicial
2.0	APO, JVL	APO, JVL	P. Cuadros Q.	2026-06-21	Actualizacion segun implementacion final del simulador
2.1	APO, JVL	APO, JVL	P. Cuadros Q.	2026-07-04	Actualizacion con version 1.8.0, CI/CD y rutas actuales

INDICE GENERAL

1. [Introduccion](#)

1.1 [Proposito](#)

1.2 [Alcance](#)

1.3 [Definiciones, Siglas y Abreviaturas](#)

1.4 [Referencias](#)

1.5 [Vision General](#)

1. [Posicionamiento](#)

2.1 [Oportunidad de negocio](#)

2.2 [Definicion del problema](#)

1. [Descripcion de los interesados y usuarios](#)

3.1 [Resumen de los interesados](#)

3.2 [Resumen de los usuarios](#)

3.3 [Entorno de usuario](#)

3.4 [Perfiles de los interesados](#)

3.5 [Perfiles de los Usuarios](#)

3.6 [Necesidades de los interesados y usuarios](#)

1. [Vista General del Producto](#)

4.1 [Perspectiva del producto](#)

4.2 [Resumen de capacidades](#)

4.3 [Suposiciones y dependencias](#)

4.4 [Costos y precios](#)

4.5 [Licenciamiento e instalacion](#)

1. [Caracteristicas del producto](#)

2. [Restricciones](#)

3. Rangos de calidad
 4. Precedencia y Prioridad
 5. Otros requerimientos del producto
- b) Estandares legales
 - c) Estandares de comunicacion
 - d) Estandares de cumplimiento de la plataforma
 - e) Estandares de calidad y seguridad
1. CONCLUSIONES
 2. RECOMENDACIONES
 3. BIBLIOGRAFIA
 4. WEBGRAFIA

1. Introduccion

1.1 Proposito

Este documento describe la vision del sistema **Simulador de Bases de Datos**, una herramienta academica para practicar consultas SQL y NoSQL, comparar sintaxis entre motores y comprender conceptos basicos de rendimiento sin instalar servidores de base de datos.

El documento sirve como referencia para comprender el problema, los usuarios, el alcance, las capacidades actuales y las restricciones del producto.

1.2 Alcance

El simulador esta orientado a estudiantes, docentes y laboratorios academicos. Permite ejecutar consultas en un entorno web/desktop, importar y exportar datos, explorar esquemas, guardar sesiones y realizar simulaciones de carga.

La version actual incluye:

- Aplicacion principal tipo IDE.
- Editor Monaco con multiples tabs.
- Siete motores simulados: SQL Server, MySQL, PostgreSQL, Oracle, SQLite, MongoDB y Redis.
- Importacion de `.sql`, `.csv` y `.json`.
- Exportacion de resultados, esquemas y bases completas.
- Persistencia local mediante IndexedDB.
- Modulo de simulacion de carga.
- Panel administrativo con monitoreo de sesiones y gestion de usuarios.
- Empaquetado web y desktop.
- Entradas separadas para aplicacion principal, simulador y administracion: `app.html`, `simulator.html` y `admin.html`.
- Validacion automatizada con GitHub Actions para rendimiento y despliegue de landing.

No incluye conexion a bases de datos reales ni ejecucion exacta de todos los dialectos de cada motor.

1.3 Definiciones, Siglas y Abreviaturas

Termino	Definicion
Motor simulado	Representacion didactica de un motor de base de datos dentro de la aplicacion.
AlaSQL	Libreria usada para ejecutar consultas SQL en memoria.
IndexedDB	Base de datos local del navegador usada para persistir tablas y esquemas.
Monaco Editor	Editor de codigo usado por Visual Studio Code, integrado en la aplicacion.
Firebase	Plataforma usada para autenticacion, presencia, roles y sesiones del simulador.
TPS	Transacciones o consultas por segundo estimadas en el simulador de carga.
Simulador de carga	Modulo que estima usuarios, latencia, CPU, conexiones y errores.
Electron	Plataforma que permite empaquetar la aplicacion web como app de escritorio.
GitHub Actions	Servicio de automatizacion usado para build, pruebas de rendimiento y despliegue.

1.4 Referencias

- Informe de Factibilidad del Proyecto Simulador de Bases de Datos (FD01), version 2.1.
- Especificacion de Requerimientos de Software del Proyecto (FD03), version 2.1.
- Documento de Arquitectura de Software del Proyecto (FD04), version 2.1.
- Documentacion oficial de React, TypeScript, Vite, Firebase, Electron, AlaSQL e IndexedDB.
- Repositorio del proyecto, scripts de construccion y workflows de GitHub Actions.

1.5 Vision General

Este documento presenta la vision del **Simulador de Bases de Datos** desde la perspectiva del problema academico, los usuarios involucrados, las capacidades principales del producto, sus restricciones y los atributos de calidad esperados. La solucion se plantea como una herramienta didactica web y desktop que permite practicar consultas SQL y NoSQL, importar datos, exportar evidencias y simular carga sin instalar motores reales.

2. Posicionamiento

2.1 Oportunidad de negocio

Aprender bases de datos suele requerir instalar motores, configurar puertos, credenciales y servicios. Esto puede dificultar el inicio para estudiantes o grupos academicos que solo necesitan practicar sintaxis, operaciones comunes y analisis de resultados.

El **Simulador de Bases de Datos** ofrece una alternativa accesible, portable y visual para practicar sin instalar SQL Server, MySQL, PostgreSQL, Oracle, SQLite, MongoDB o Redis por separado.

2.2 Definicion del problema

Los estudiantes pueden enfrentar dificultades para:

- Instalar varios motores de base de datos en un mismo equipo.
- Comprender diferencias de sintaxis entre motores.
- Probar consultas sin riesgo de afectar una base real.

- Exportar evidencias de resultados.
- Simular carga o saturacion sin infraestructura.
- Mantener sesiones de trabajo reutilizables.

El proyecto aborda este problema mediante un entorno integrado, local y academico.

3. Descripcion de los interesados y usuarios

3.1 Resumen de los interesados

Actor	Descripcion	Necesidad
Docente	Evalua el aprendizaje y el producto.	Ver evidencias, documentacion y comportamiento funcional.
Desarrollador	Mantiene y mejora el proyecto.	Modificar componentes, motores simulados y reglas de exportacion.
Administrador	Gestiona usuarios y observa sesiones del simulador.	Monitorear actividad y asignar roles.
Equipo de desarrollo	Responsable de construir y mantener la solucion.	Contar con una base modular, portable y documentada.

3.2 Resumen de los usuarios

Usuario	Descripcion	Necesidad
Estudiante	Usuario principal del sistema.	Practicar consultas, importar datos y revisar resultados.
Usuario de laboratorio	Usa el simulador en ejercicios guiados.	Ejecutar pruebas sin instalar motores reales.
Administrador	Usuario con privilegios de gestion.	Monitorear sesiones y gestionar roles.

3.3 Entorno de usuario

El sistema se usa desde navegadores modernos y, opcionalmente, como aplicacion desktop empaquetada con Electron. En modo web, el usuario accede a la aplicacion principal, al simulador independiente o al panel administrativo mediante las entradas `app.html`, `simulator.html` y `admin.html`.

Los datos de practica se almacenan localmente en el navegador mediante IndexedDB. Las funciones de autenticacion, presencia, roles y monitoreo requieren configuracion de Firebase.

3.4 Perfiles de los interesados

- **Docente:** revisa evidencias, documentacion, cumplimiento de requisitos y comportamiento funcional del sistema.
- **Desarrollador:** mantiene componentes React, motores simulados, persistencia local, servicios Firebase y flujos de exportacion.
- **Administrador:** supervisa usuarios y sesiones cuando el entorno Firebase esta configurado.

3.5 Perfiles de los Usuarios

- **Estudiante:** escribe consultas, importa datos, visualiza resultados, exporta evidencias y ejecuta simulaciones de carga.

- **Usuario de laboratorio:** sigue practicas guiadas y usa el sistema sin instalar motores reales.
- **Usuario administrador:** accede al panel administrativo para monitorear actividad y asignar roles.

3.6 Necesidades de los interesados y usuarios

- Practicar SQL y NoSQL en un entorno integrado.
- Comparar motores y sintaxis sin instalar servidores externos.
- Importar archivos `.sql`, `.csv` y `.json`.
- Exportar resultados, esquemas, bases completas y logs.
- Conservar historial y evidencias de aprendizaje.
- Simular carga, latencia, errores y saturacion de manera didactica.
- Mantener separada la simulacion academica de cualquier base de datos real.

4. Vista General del Producto

4.1 Perspectiva del producto

El producto funciona como una aplicacion web con posibilidad de empaquetado desktop. La interfaz principal ejecuta consultas en el navegador, guarda datos locales en IndexedDB y usa Firebase para funciones de autenticacion y monitoreo cuando existe configuracion.

Las entradas principales son scripts SQL, comandos MongoDB, comandos Redis y archivos importados. Las salidas principales son tablas de resultados, mensajes, exportaciones y metricas simuladas.

4.2 Resumen de capacidades

Capacidad	Descripcion
Editor de consultas	Permite escribir, seleccionar y ejecutar codigo.
Multi-motor	Ofrece tabs para SQL Server, MySQL, PostgreSQL, Oracle, SQLite, MongoDB y Redis.
Importacion	Carga scripts SQL, CSV y JSON para crear tablas.
Exportacion	Descarga resultados en CSV, JSON, Excel, DDL y formatos por motor.
Explorador de esquema	Muestra bases registradas, tablas, columnas y previsualizacion.
Historial	Guarda consultas y bitacoras de ejecucion.
Simulacion de entorno	Configura latencia, errores, aislamiento y limite de conexiones.
Simulacion de carga	Mide TPS, latencia, errores, CPU estimada y saturacion.
Administracion	Permite monitorear usuarios y sesiones activas.
Desktop	Puede ejecutarse como aplicacion Electron.
CI/CD	Ejecuta validaciones de rendimiento y despliega la landing desde GitHub Actions.

4.3 Suposiciones y dependencias

- El usuario cuenta con un navegador moderno o con la version desktop generada con Electron.
- La ejecucion SQL depende de AlaSQL y de los preprocesadores implementados por motor.
- MongoDB y Redis son simulaciones internas, no conexiones a servicios reales.

- La persistencia local depende de IndexedDB y del almacenamiento del navegador.
- Firebase debe configurarse para autenticacion, presencia, roles y monitoreo.
- Los workflows de GitHub Actions dependen del repositorio y de la configuracion de CI/CD.

4.4 Costos y precios

El proyecto no requiere licencias comerciales para su ejecucion academica. Las herramientas principales son open-source o tienen niveles gratuitos suficientes. Los costos considerados se relacionan principalmente con tiempo academico de desarrollo, internet, energia, materiales y documentacion, detallados en el FD01.

4.5 Licenciamiento e instalacion

La version actual define tres entradas principales en `vite.config.ts`:

Entrada	Uso
<code>app.html</code>	IDE principal del simulador.
<code>simulator.html</code>	Simulador de carga independiente.
<code>admin.html</code>	Panel administrativo.

Para despliegue, el proyecto usa `netlify.toml` con rutas limpias `/app`, `/simulador` y `/admin`, y el workflow `Deploy Landing Page` para publicar la landing estatica alojada en `landing/`.

El uso recomendado en desarrollo requiere instalar dependencias con `npm install` y ejecutar `npm run dev`. Para escritorio, el sistema puede generarse mediante los scripts Electron definidos en el proyecto.

5. Caracteristicas del producto

5.1 Motores soportados

La version actual incluye:

- `sqlserver`
- `mysql`
- `postgresql`
- `oracle`
- `sqlite`
- `mongodb`
- `redis`

Cada motor cuenta con configuracion visual, conexion ficticia, factor de rendimiento para simulacion y tratamiento especifico de comandos o sintaxis.

5.2 Aplicacion principal

La aplicacion principal incluye:

- Login y registro.
- Pantalla de bienvenida.
- Barra superior con acciones de ejecutar, importar, exportar, ayuda, estadisticas, historial y simulador.

- Sidebar con herramientas.
- Editor SQL/NoSQL.
- Panel de resultados.
- Explorador de esquema.
- Selector de entorno y estado de conexión simulado.

5.3 Simulador de carga

El simulador de carga permite:

- Seleccionar motor.
- Definir duración, usuarios máximos y rampa.
- Elegir tipos de consulta: SELECT, INSERT, UPDATE y DELETE.
- Simular latencia, CPU, conexiones, TPS y errores.
- Comparar dos motores.
- Ejecutar modo progresivo hasta detectar saturación.
- Guardar historial de pruebas.
- Exportar logs en JSON y CSV.

5.4 Panel administrativo

El panel administrativo incluye:

- Login de administrador.
- Monitoreo de sesiones activas.
- KPIs de usuarios conectados, sesiones corriendo, TPS promedio y motores activos.
- Tabla de actividad por usuario.
- Gestión de roles de usuarios.

6. Restricciones

- La aplicación no se conecta a motores reales.
- La compatibilidad SQL depende de AlaSQL y de los preprocesadores implementados.
- MongoDB y Redis son simulaciones en memoria.
- La persistencia principal de datos de usuario es local al navegador.
- Las funciones colaborativas requieren variables de entorno de Firebase.
- Los resultados de rendimiento son simulados y no deben usarse como benchmark real.
- El build desktop depende de Electron y del sistema operativo objetivo.

7. Rangos de calidad

Atributo	Descripcion
Usabilidad	Interfaz tipo IDE con acciones visibles, modales y plantillas.
Mantenibilidad	Separacion entre componentes, store, motores, librerias y datos.
Portabilidad	Ejecucion web y empaquetado desktop.
Auditabilidad	Historial, logs de consulta y exportaciones.
Configurabilidad	Ajustes de editor, simulacion y entorno.
Disponibilidad local	La aplicacion puede ejecutarse en desarrollo con Vite.
Seguridad basica	Autenticacion y roles mediante Firebase cuando esta configurado.

8. Precedencia y Prioridad

La prioridad del producto se centra en las funciones que permiten cumplir el objetivo academico principal:

1. Ejecucion local de consultas SQL y operaciones NoSQL simuladas.
2. Importacion, persistencia local y exploracion de datos.
3. Exportacion de resultados, esquemas y evidencias.
4. Simulacion de carga y comparacion didactica entre motores.
5. Panel administrativo, monitoreo y roles con Firebase.
6. Empaquetado desktop y automatizacion CI/CD.

9. Otros requerimientos del producto

b) Estandares legales

El sistema no debe almacenar datos sensibles reales por defecto. Si se importan datos reales en ejercicios academicos, el usuario debe contar con autorizacion y respetar las normas de proteccion de datos aplicables.

c) Estandares de comunicacion

La documentacion debe comunicar claramente que el sistema es un simulador academico y que no reemplaza motores reales ni clientes profesionales de administracion de bases de datos.

d) Estandares de cumplimiento de la plataforma

La aplicacion debe ejecutarse en navegadores modernos y conservar compatibilidad con el empaquetado desktop mediante Electron. Las rutas web deben mantenerse coherentes con `app.html`, `simulator.html`, `admin.html`, `/app`, `/simulador` y `/admin`.

e) Estandares de calidad y seguridad

El sistema debe mantener separacion modular entre interfaz, estado, motores, persistencia y servicios. Las credenciales de Firebase deben permanecer fuera del repositorio publico y las funciones administrativas deben depender de autenticacion y roles.

CONCLUSIONES

El **Simulador de Bases de Datos** cumple la vision de una herramienta academica para practicar consultas, explorar diferencias entre motores y simular condiciones de carga. La version actual es mas completa que un editor simple porque integra importacion, exportacion, persistencia local, simulacion de rendimiento, administracion, despliegue web/desktop y validacion automatizada con GitHub Actions.

RECOMENDACIONES

1. Mantener visible en la documentacion que los motores son simulados.
2. Agregar ejemplos guiados por motor para estudiantes.
3. Documentar la configuracion de Firebase para funciones administrativas.
4. Ampliar pruebas automatizadas para importadores, exportadores y motor SQL.
5. Conservar la separacion entre simulacion academica y futuras conexiones reales.

BIBLIOGRAFIA

- Sommerville, I. (2016). *Software Engineering*.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts*.
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems*.

WEBGRAFIA

- React Documentation: <https://react.dev/>
- TypeScript Documentation: <https://www.typescriptlang.org/docs/>
- Vite Documentation: <https://vitejs.dev/>
- Firebase Documentation: <https://firebase.google.com/docs>
- Electron Documentation: <https://www.electronjs.org/docs/latest/>